Stable and Safe Neural ODEs

Tianyi Xia
School of Engineering and Applied Science
University of Pennsylvania

Aditya Vats
School of Engineering and Applied Science
University of Pennsylvania

1. Motivation

Learning from Demonstration (LfD) lets a non-expert user teach a robot a skill by physically guiding it or recording kinesthetic trajectories [1]. A central design choice is the function class used to represent the learned policy. Dynamical-System (DS) representations are attractive because they yield a time-invariant velocity field $\dot{x} = f(x)$ that can be re-executed from any initial state, and that responds smoothly to perturbations during execution. Classical formulations such as SEDS [2] and LPV-DS [3] put a parametric structure on f and solve a constrained optimization problem in order to obtain a Lyapunov-stable system. Recently, *Neural Ordinary Differential Equations* (NODEs) [4] have been proposed as a much more expressive function class for the same problem. [9] proposed a stability guarantee on vanilla NODE. Nawaz et al. [7] apply a CBF-QP filter at inference time on top of a learned NODE. Xiao et al. [6] solve a QP on the parameters of the network; Miao et al. [8] embed a differentiable QP layer into the forward pass.

Our motivation is to elevate prior research in Neural ODEs by proposing a novel framework that ensures rigorous safety guarantees while maintaining Lyapunov stability and imitation accuracy. Grounded in a comprehensive understanding of prior methodologies, we aim to advance the current state-of-the-art and comprehensively validate our approach through extensive experimental evaluations, demonstrating stable imitation and reliable obstacle avoidance. The implementation for the same is available at https://github.com/Skylar-xty/InvarianceNODE/tree/final project

2. Goal

Given demonstrations $\{(x_i, \dot{x}_i)\}_{i=1}^{N}$ collected from human trajectories that converge to a target $x^\star$, together with a list of known obstacles $\mathcal{O} = \{(c_j, r_j)\}_{j=1}^{M}$, our goal is to *jointly* learn

$$f_\theta : \mathbb{R}^D \to \mathbb{R}^D, \qquad V_\phi : \mathbb{R}^D \to \mathbb{R}_{\geq 0}$$

such that the closed-loop system $\dot{x} = f_\theta(x)$

1) **Imitates** the demonstrations, i.e. rolling out from a demonstration's initial condition tracks $\dot{x}_i$.
2) Is **stable** in the Lyapunov sense to the attractor $x^\star$:
$$\nabla V_\phi \cdot f_\theta + \alpha\, V_\phi \;\leq\; 0, \qquad \alpha > 0.$$
3) Is **forward invariant** with respect to the safe set $\bigcap_j \{x : h_j(x) \geq 0\}$, where each h_j describes an obstacle:
$$\nabla h_j \cdot f_\theta + \alpha_h\, h_j \;\geq\; 0, \qquad \forall j.$$

3. Approach

3.1. Neural ODE policy

The vector field is parameterized by a Multi-Layer Perceptron (MLP)

$$f_\theta : \mathbb{R}^D \to \mathbb{R}^D,$$

with hidden width 128 and $\tanh$ activations. We use the residual variant $f_\theta(x) = -\beta\,(x - x^\star) + g_\theta(x)$ in 3D experiments to bias the field toward the attractor; the 2D framework uses the plain MLP. The closed-loop semantics is

$$\dot{x} = f_\theta(x), \qquad x(0) \in \mathbb{R}^D.$$

3.2. Stability via Warped-ICNN Lyapunov and Control Lyapunov Function

We adopt a learned Lyapunov function in the spirit of [9], but with a residual warping network F in front of the Input-Convex Neural Network g [10]:

$$V_\phi(x) \;=\; \sigma_{k+1}\big(g(F(x)) - g(F(0))\big) \;+\; \epsilon\,\|x\|_2^2, \qquad (1)$$

where F is an unconstrained MLP, g is an ICNN with two hidden layers of width 64, and σ_{k+1} is a smoothed-ReLU outer activation. The extra $\epsilon\|x\|^2$ term enforces strict positive-definiteness around the goal and convexity-relaxation enforces $V_\phi \geq 0$ everywhere. By construction $V_\phi(x^\star) = 0$.

The classical CLF condition $\nabla V \cdot f + \alpha V \leq 0$ is *linear* in f. Its feasible set in $\mathbb{R}^D$ is therefore a half-space, and the orthogonal projection of any candidate vector $\hat{f}$ onto it is in closed form:

$$\pi_{\mathrm{CLF}}(\hat{f}) \;=\; \hat{f} \;-\; \nabla V\, \frac{\mathrm{ReLU}\big(\nabla V \cdot \hat{f} + \alpha V\big)}{\|\nabla V\|^2}. \qquad (2)$$

∇V is obtained via autograd through the differentiable V_ϕ.

3.3. Safety

3.3.1. QP-based safety. The classical Ames-style construction [5] solves

$$\begin{aligned} f^\star(x) = \arg\min_{f,\,\delta} \;& \tfrac{1}{2}\|f - \hat{f}(x)\|^2 + p\,\delta^2 \\ \text{s.t. } & \nabla h_j(x)\cdot f + \alpha_h\, h_j(x) \geq 0,\ \forall j \\ & \nabla V_\phi(x)\cdot f + \alpha_{V_\phi} V_\phi(x) \leq \delta \\ & \delta \geq 0. \end{aligned} \tag{3}$$

where $\hat{f}(x)$ is the nominal vector field (e.g., the raw NODE output), $V_\phi(x)$ is a Control Lyapunov Function, $h_j(x)$ are Control Barrier Functions for each obstacle, $\alpha_V, \alpha_h > 0$ are class-$\mathcal{K}$ gains, $p > 0$ is the slack penalty weight, and $\delta \geq 0$ is the CLF slack. The closed-loop is $\dot{x} = f^\star(x)$.

Following [6], this can be re-written as a QP on the parameters of the network's last layer by treating $\theta^P_{K-1,K}$ as a virtual control: the CBF and CLF constraints in parameter space read

$$\begin{aligned} &\tfrac{\partial h}{\partial x}\,\theta^P_{K-1,K} z^P_{K-1} + \tfrac{\partial h}{\partial x}\,\theta^N_{K-1,K} z^N_{K-1} + \alpha_h h(x) \geq 0, \\ &\tfrac{\partial V}{\partial x}\,\theta^P_{K-1,K} z^P_{K-1} + \tfrac{\partial V}{\partial x}\,\theta^N_{K-1,K} z^N_{K-1} + \alpha_V V(x) \leq \delta. \end{aligned}$$

In our two-stage QP pipeline we first train (f_θ, V_ϕ) jointly without obstacles, then freeze them and train a class-$\mathcal{K}$ function for the CBF using a differentiable QP.

3.3.2. Analytic projection. We model each obstacle as a bounded ellipsoid:

$$h_j(x) \;=\; \sum_{i=1}^{D} \Big(\frac{x_i - c_{j,i}}{r_{j,i}}\Big)^2 - 1. \tag{4}$$

The CBF condition is again linear in f. The closed-form projection is

$$\pi_{\text{CBF}}(f) \;=\; f \;+\; \sum_{j=1}^{M} \nabla h_j\, \frac{\text{ReLU}\big(-(\nabla h_j \cdot f + \alpha_h\, h_j)\big)}{\|\nabla h_j\|^2}. \tag{5}$$

The summation is the simultaneous correction toward each violated half-space; we found that, for the LASA configurations we test, $M \leq 4$ and a single summation step suffices because each ∇h_j is well separated.

A standard concern with sequential half-space projections is that one projection can re-introduce violation of the other. We follow the Projection Onto Convex Sets (POCS) tradition [13], [14] and *alternate* the two projections, as summarized in Algorithm 1.

Theorem 3.1 (POCS rate)**.** *Let $\theta(x) = \angle(\nabla V(x), \nabla h(x))$ and assume $\mathcal{H}_V \cap \mathcal{H}_h \neq \emptyset$. The iterates $f^{(k)}$ generated by Algorithm 1 satisfy*

$$\big\|f^{(2n_{\text{iter}})} - f^\star\big\| \;\leq\; \cos^{2n_{\text{iter}}}\big(\theta(x)\big)\,\big\|\hat{f} - f^\star\big\|,$$

where $f^\star = P_{\mathcal{H}_V \cap \mathcal{H}_h}(\hat{f})$ is the joint-feasible projection.

Each CLF projection drives f onto the boundary of $\mathcal{H}_V$, after which the CBF step adds a ∇h-aligned correction that generally re-introduces a small CLF residual unless $\nabla V \perp \nabla h$. Standard POCS results [12] give the contraction rate.

On the LASA configurations we test, the angle between ∇V and ∇h exceeds 45° at over 95% of demo points, so n_{iter}=2 brings the residual below $0.25 \cdot \|\hat{f} - f^\star\|$, in agreement with the empirically observed $<1\%$ CLF residual at the projection output. We therefore use n_{iter}=2 as the default. Because the last operation in Algorithm 1 is always the CBF step, the output satisfies the CBF condition *exactly* for any $n_{\text{iter}} \geq 1$ — CLF feasibility is the only quantity that benefits from more iterations.

Algorithm 1 SAFENEURALODE forward pass

```
function FORWARD(x; θ, φ, x*, O, α, α_h, h_cap, n_iter)
    f ← f_θ(x)                                   ▷ raw network output
    for k = 1, ..., n_iter do
        f ← π_CLF(x, f, V_φ, x*, α)
        f ← π_CBF(x, f, O, α_h, h_cap)
    end for
    return f
end function
```

3.3.3. HJB reachability barrier. Hamilton Jacobi Reachability definition:

$$V(x,t) = \min_{\tau\in[0,t)} \max_u h(x(\tau)) \tag{6}$$

For ∞ time, V is stationary. $V(x) = \min_{\tau\in[0,\infty)} \max_u h(x(\tau))$.

Few references for ∞ time: $V(x,\infty)$ has undesirable properties like discontinuities, finiteness and flatness. They try to solve it using HJ-PDE: $min[h(x) - V(x), \nabla V(x)\dot{x} + \gamma V(x)] = 0$ with boundary conditions but it is difficult to solve using general learning techniques. **Forward Invariance is ensured by definition if V(x)$\geq$ 0**

- Simplification 1: No control variable $\rightarrow$ $V(x) = \min_{\tau\in[0,\infty)} h(x(\tau))$
- Simplification 2: Lyapunov Exponential Stability constraint $\implies x^*$ is equilibrium point $\implies$ After finite time, trajectory goes and stops η close to x^*. Therefore:

$$V(x,\infty) \rightarrow V(x, T_{fin}) = \min_{\tau\in[0,T_{fin})} h(x(\tau)) \tag{7}$$

- **Insight : Show invariance for T_{fin}, invariance forever**

Value function to learn: $V_{hjb}(x,\infty) \rightarrow V_{hjb}(x, T_{fin}) = \min_{\tau\in[0,T_{fin}]} h(x(\tau))$

3.4. Training

We train f_θ and V_ϕ jointly on the demonstration set $\{(x_i, \dot{x}_i)\}_{i=1}^N$ with attractor $x^\star$ and obstacle set $\mathcal{O} = \{(c_j, r_j)\}_{j=1}^M$. The total loss combines an imitation term

through the projection with two projection-correction penalties, a kNN velocity-copy term, and an optional Lyapunov-smoothness term:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_V \mathcal{L}^V_{\text{proj}} + \lambda_H \mathcal{L}^h_{\text{proj}} + \lambda_{vc} \mathcal{L}_{vc} + \lambda_{sm} \mathcal{L}_{\text{smooth}}. \tag{8}$$

The individual terms are summarized in Table 1. Two design choices deserve emphasis. First, $\mathcal{L}_{\text{task}}$ is evaluated on the *projected* output $f_{\text{proj}} = \pi_{\text{CBF}} \circ \pi_{\text{CLF}}(\hat{f})$, so the network is trained to imitate demonstrations through the same projection it will use at inference time; and there is no train/test mismatch. Second, the two penalties $\mathcal{L}^V_{\text{proj}}, \mathcal{L}^h_{\text{proj}}$ shape the raw field $\hat{f}$ to already satisfy the CLF/CBF conditions, so the analytic projection acts as a near-identity correction at convergence and the POCS rate of Theorem 3.1 is exercised in a benign regime.

TABLE 1: Loss terms used in (8).

Term	Definition
$\mathcal{L}_{\text{task}}$	$\frac{1}{N}\sum_i \|f_{\text{proj}}(x_i) - \dot{x}_i\|^2$
$\mathcal{L}^V_{\text{proj}}$	$\mathbb{E}_{x\sim\mathcal{P}_V}[\|\Delta f_{\text{CLF}}(\hat{f}(x))\|^2]$
$\mathcal{L}^h_{\text{proj}}$	$\mathbb{E}_{x\sim\mathcal{P}_H}[\sum_j \|\Delta f_{\text{CBF},j}(\hat{f}(x))\|^2]$
$\mathcal{L}_{vc}$	$\mathbb{E}_{x\sim\mathcal{P}_V}[w_\sigma(x)\,\|\hat{f}(x) - \dot{x}_{\text{nn}(x)}\|^2]$
$\mathcal{L}_{\text{smooth}}$	$\mathbb{E}_i[\text{ReLU}(V_\phi(x_i + \epsilon\dot{x}_i) - V_\phi(x_i))]$

Here $\Delta f_{\text{CLF}}, \Delta f_{\text{CBF},j}$ are the single-step corrections of Eq. (2) and Eq. (5), and $\dot{x}_{\text{nn}(x)}$ is the demo velocity at the nearest demo point to x, weighted by a Gaussian kernel $w_\sigma(x) = \exp(-\|x - x_{\text{nn}}\|^2/\sigma^2)$. $\mathcal{L}_{vc}$ supplies an explicit directional signal on the tube around demos, which is needed because $\mathcal{L}^V_{\text{proj}}$ alone only enforces CLF-feasibility (a half-space constraint), leaving a continuum of distinct fields all of which can produce "shortcut" rollouts that bypass the demo geometry.

3.4.1. Off-demo sampling. Each epoch we draw two small off-demo batches that supplement the demonstration points in the penalty losses.

The CLF batch is sampled uniformly in the demo bounding box (inflated by a fixed margin). If a tube radius ρ is given, we oversample $4\times$ and keep only points within distance ρ of the nearest demo point, i.e. a tube around the demonstration manifold; if $\rho = \infty$, we just keep the full bounding-box sample. This batch serves two losses at once: the CLF feasibility penalty $\mathcal{L}^V_{\text{proj}}$, which asks the raw network $\hat{f}_\theta$ to already satisfy the CLF inequality at these points (so the projection acts as identity there); and a k-NN velocity-copy term, which softly pushes $\hat{f}_\theta(x)$ toward the velocity of the nearest demo point, weighted by a Gaussian in distance. The tube confines both forms of supervision to the manifold the demos actually traverse, which prevents the network from discovering shortcuts on the opposite side of an obstacle from the demonstrations.

The CBF batch is drawn independently for each obstacle: uniform inside an axis-aligned box of half-side $1.2\,r_j$ centred at c_j (so the box just envelops the obstacle's normalized unit sphere with a 20% margin), then concatenated across obstacles. The CBF feasibility penalty $\mathcal{L}^h_{\text{proj}}$ is evaluated on this batch plus the demonstration points. Keeping the box tight matches the h_{cap} region where the projection is actually active, so CBF pressure stays local and $\hat{f}_\theta$ is undistorted in the bulk of the workspace.

3.4.2. Training variants. The same loss (8) is used in all three safety formulations of Section 3.3, with only the safety layer changing:

1) **QP-based safety**. Two-stage: first train (f_θ, V_ϕ) jointly with $\lambda_H = 0$ (no obstacles), then freeze them and learn a class-$\mathcal{K}$ gain $\alpha_h(x)$ for each CBF through a differentiable QP. At inference, the CLF+CBF QP is solved at every integration step.
2) **Analytic projection**. Single stage: the projection layer is wired into the forward pass and the full loss (8) is back-propagated end-to-end. Inference is plain Euler over f_{proj}, no QP solver needed.
3) **HJB reachability barrier**. A value-function network is fit with an MSE loss against a grid solution of the HJB PDE; the learned V_{hjb} replaces h inside the analytic-projection variant above without changing the training machinery.

3.5. Residual safety correction

The single-stage training of Section 3.4 couples three objectives—demo imitation, CLF compliance, and CBF compliance—into one loss, and pays for it with seven penalty-shaping knobs (Sections 3.4.1). We additionally propose a two-stage *residual* variant that decomposes stability and safety into separately trained networks, at the cost of a second optimization stage.

3.5.1. Architecture. We replace the single network f_θ with a frozen *base* imitator $f_B(x;\theta^\star_B)$ together with a small *residual* $\Delta f_\psi(x;\psi)$, gated by a smooth indicator $\omega(x)$:

$$f(x) \;=\; \pi_{\text{CBF}}\Big(\pi_{\text{CLF}}\big(f_B(x) + \omega(x)\cdot\Delta f_\psi(x)\big)\Big), \tag{9}$$

with

$$\omega(x) \;=\; \mathbf{1}\big[h_{\min}(x) < h_{\text{cap}}\big]\cdot\sigma\big(k\,(h_{\text{cap}} - h_{\min}(x))\big), \tag{10}$$

where $h_{\min}(x) = \min_j h_j(x)$ is the value of the closest barrier, σ is the logistic sigmoid, and k controls how sharply ω transitions from 1 inside the cap region to 0 outside. Δf_ψ is a two-layer MLP with hidden dimension 64 and a bounded $\tanh$ output ($\|\Delta f_\psi\|_\infty \le 2$), so a poorly-trained residual cannot produce arbitrarily large deflections; the analytic projection layer is still the by-construction safety net.

3.5.2. Two-stage training. *Stage 1* trains (f_B, V_ϕ) jointly using the loss (8) *without obstacles* ($\lambda_H = \lambda_{vc} = 0, \mathcal{O} = \emptyset$). This produces a clean CLF-only imitator whose closed loop converges to $x^\star$ from any initial condition in the workspace. Stage 1 reuses `train_inline.py` unchanged. *Stage 2* freezes $\theta^\star_B, \phi^\star$ and trains only ψ, on the same demonstrations but with the obstacles present. The Stage-2 loss is

identical to (8) except λ_V and λ_{vc} are set to zero (the CLF behaviour is already locked in by f_B), leaving an imitation term through the residual-augmented projection together with the CBF correction penalty $\mathcal{L}^h_{\text{proj}}$ and a mild ℓ_2 regularizer on Δf_ψ.

3.5.3. Outside-cap exactness. The smooth indicator ω is gated by the same threshold h_{cap} that bounds the barrier $\tilde{h}_j = \min(h_j, h_{\text{cap}})$. As a consequence, whenever $h_{\min}(x) \geq h_{\text{cap}}$ we have both $\omega(x) = 0$ *and* $\nabla \tilde{h}_j(x) = 0$ for every obstacle. Substituting into (9):

$$h_{\min}(x) \geq h_{\text{cap}} \implies f(x) = \pi_{\text{CLF}}\big(f_B(x)\big), \qquad (11)$$

i.e. the v3 closed loop is *identical* to the Stage-1 closed loop outside the cap region. CLF stability and the basin of attraction estimated for (f_B, V_ϕ) in Stage 1 are inherited verbatim — there is no approximation, no perturbation argument, just the bounded barrier and the indicator both vanishing. Inside the cap region, the residual fires and the CBF projection is non-trivial, but safety remains exact by construction of π_{CBF}.

3.5.4. Tradeoffs. Relative to the joint single-stage variant of Section 3.4, the residual design buys

1) the strict outside-cap statement (11),
2) a sharply reduced hyperparameter surface (Stage 2 has four knobs: $h_{\text{cap}}, k, \|\Delta f_\psi\|_\infty$, and the residual hidden size),
3) modular reuse of one base imitator across different obstacle configurations. It pays one extra MLP evaluation per integration step and gives up the ability to globally re-shape f_B when demonstrations themselves need rerouting around obstacles that the no-obstacle Stage 1 ignored.

4. Evaluation

We evaluate the framework on the LASA handwriting dataset [2], which provides 26 two-dimensional human-drawn shapes with associated time-derivatives, plus 3D extension. Throughout, we report (i) imitation quality, (ii) stability of long rollouts, and (iii) safety with respect to obstacles. The full evaluation follows.

4.1. Imitation quality (no obstacle): vs. LPV-DS

We compare the analytic-projection NODE against the LPV-DS baseline of Figueroa and Billard [3] (constraint variant O3, P-QLF Lyapunov, PC-GMM mixing) trained on identical demonstrations (7 trajectories, sub-sample $=$ 2, `ds-opt/phys-gmm` default hyper-parameters). Three LASA shapes of increasing geometric difficulty are reported: *BendedLine* (long, gentle bend), *Leaf_2* (two lobes, mild curvature reversal), and *Snake* (high-frequency oscillations).

For every shape we report four metrics computed in raw LASA position units (mm), all matching the convention of `ds-opt`: DTWD with band $=$ 20 as a shape-similarity measure between reproductions and demonstrations; RMSE of velocity and the cosine-style deviation $\dot{e} = 1 - \cos\angle(f_\theta(x), \dot{x})$ on every demo data point; and the trajectory imitation MSE (only computed on our side, since LPV-DS does not report it). The corresponding qualitative overlays — streamlines, rollouts from each demo's initial condition, and Lyapunov contours — are shown alongside in Figs. 1, 2, and 3.

Across all three shapes our approach attains a uniform improvement on every metric. The qualitative gap visible in the figures is twofold:

1) The streamlines of our model reproduce the demo manifold even far from the data, where LPV-DS streamlines pull toward the attractor along the principal axis of the learned P;
2) Our learned Lyapunov function $V_\phi(x)$ has contours that bend along the demonstration geometry, whereas the LPV-DS $V(x) = (x - x^\star)^\top P (x - x^\star)$ is constrained to nested ellipsoids.

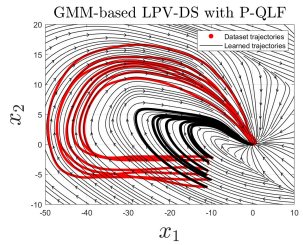
The latter is the structural explanation for the gap: a quadratic Lyapunov candidate restricts the admissible A_k to those whose flow descends a fixed ellipsoid, biasing the vector field away from highly-curved demo geometry.

4.1.1. BendedLine. Despite being the geometrically simplest of the three motions, *BendedLine* exposes the most dramatic quantitative gap (Table 2): the LPV-DS DTWD is 10287 ± 1083 versus our 2040 ± 1558, a $5.0\times$ reduction in shape error. Velocity RMSE drops by 49% and the directional deviation $\dot{e}$ by $5.8\times$. The cause is visible in Fig. 1: LPV-DS streamlines bow sharply toward the attractor along the long axis of the ellipsoidal P, so the gentle right-to-left bend of the demo is squashed into a near-linear contraction. Our V_ϕ contours, by contrast, follow the bend itself, leaving the f_θ free to interpolate the demos almost exactly. The long, slowly-varying trajectory amplifies any directional mis-alignment, which is why a moderate visual gap produces such a large DTWD difference.

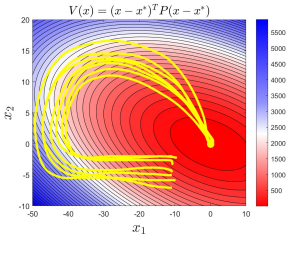
TABLE 2: Quantitative comparison on the LASA *BendedLine* motion (7 demos, raw mm-scale units).

Metric	LPV-DS (O3) [3]	Stable NODE (ours)
DTWD (total, band $=20$)	10287.26 ± 1083.20	$\mathbf{2040.24 \pm 1557.53}$
RMSE of velocity [mm/s]	3.01	$\mathbf{1.55}$
Mean velocity deviation $\dot{e}$	4.73×10^{-2}	$\mathbf{0.81\times10^{-2}}$

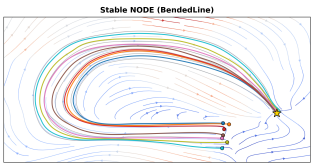
4.1.2. Leaf_2. *Leaf_2* contains a mild curvature reversal that already exceeds the descriptive power of a quadratic Lyapunov function. On Table 3 our method improves DTWD by $3.4\times$ ($4988 \rightarrow 1479$), velocity RMSE by 35%, and $\dot{e}$ by $5.5\times$. The qualitative panel (Fig. 2, top-right) shows the LPV-DS Lyapunov function as a broad ellipse



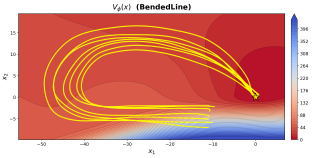
(a) LPV-DS: vector field with reproductions.



(b) LPV-DS: quadratic Lyapunov function $V(x) = (x - x^*)^\top P(x - x^*)$.



(c) Stable NODE (ours): vector field with reproductions.



(d) Stable NODE (ours): learned Lyapunov function $V_\phi(x)$.

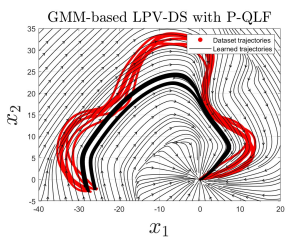
Figure 1: Qualitative comparison on the LASA *BendedLine* motion. Top row: LPV-DS [3]; bottom row: ours. Left column shows the learned vector field with rollouts from the demonstration initial conditions; right column shows the corresponding Lyapunov function.

aligned with the lobe average, which forces a single contracting direction and explains why the LPV-DS streamlines in the top-left panel sag toward the attractor instead of tracing the two lobes. Our learned V_ϕ (bottom-right) develops asymmetric, gently curved level sets that respect the lobe structure, and the corresponding rollouts (bottom-left) hug the demos throughout the motion. The standard deviation of our DTWD (± 1272) is dominated by one outlier rollout that cuts across the upper lobe; the remaining six are within ~ 1500 of the demos.

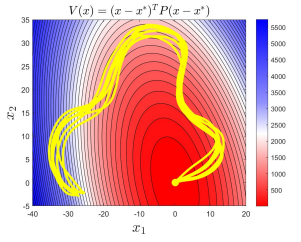
TABLE 3: Quantitative comparison on the LASA *Leaf_2* motion (7 demos, raw mm-scale units).

Metric	LPV-DS (O3)	Stable NODE (ours)
DTWD (total, band = 20)	4988.26 ± 443.75	$\mathbf{1478.78 \pm 1271.63}$
RMSE of velocity [mm/s]	4.47	**2.90**
Mean velocity deviation $\dot{e}$	8.16×10^{-2}	$\mathbf{1.46\times10^{-2}}$

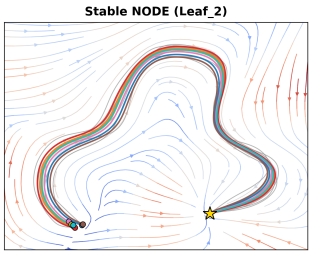
4.1.3. Snake. *Snake* rapidly alternates curvature with three sign changes in $\dot{x}$. The LPV-DS DTWD (3609 ± 313) is relatively low because the motion is short, but velocity errors are the highest in absolute terms (RMSE $= 3.30$ mm/s, $\dot{e} = 3.9\times10^{-2}$); on top of that, the LPV-DS streamlines (Fig. 3, top-left) visibly average out the oscillations, so individual rollouts cut across the snake's coils rather than following them. Our model (Table 4) reduces DTWD by 29%, RMSE by 24%, and $\dot{e}$ by $3.3\times$; more importantly, the rollouts in Fig. 3 (bottom-left) preserve every turn of the snake. The improvement is smaller *in DTWD* than for



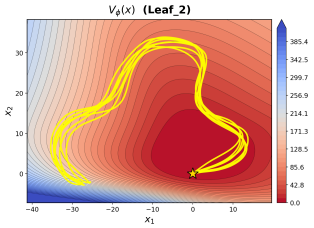
(a) LPV-DS: vector field with reproductions.



(b) LPV-DS: quadratic Lyapunov function $V(x) = (x - x^*)^\top P(x - x^*)$.



(c) Stable NODE (ours): vector field with reproductions.



(d) Stable NODE (ours): learned Lyapunov function $V_\phi(x)$.

Figure 2: Qualitative comparison on the LASA *Leaf_2* motion. Top row: LPV-DS; bottom row: ours. Left column shows the learned vector field with rollouts from the demonstration initial conditions; right column shows the corresponding Lyapunov function.

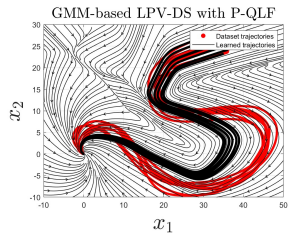
BendedLine because the high-frequency component of the demo contributes to DTWD almost regardless of macroscopic alignment; the velocity-direction metric $\dot{e}$ is therefore the more informative indicator on this shape and shows the largest relative gain.

TABLE 4: Quantitative comparison on the LASA *Snake* motion (7 demos, raw mm-scale units).

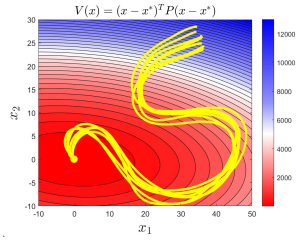
Metric	LPV-DS (O3) [3]	Stable NODE (ours)
DTWD (total, band = 20)	3608.84 ± 312.52	$\mathbf{2544.91 \pm 1125.28}$
RMSE of velocity [mm/s]	3.30	**2.51**
Mean velocity deviation $\dot{e}$	3.88×10^{-2}	$\mathbf{1.18\times10^{-2}}$

4.2. Obstacle avoidance via analytic projection

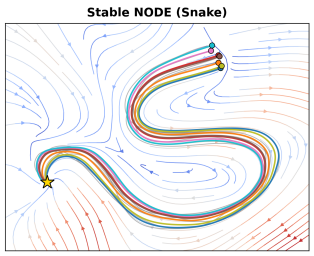
We place one ellipsoidal obstacle on top of the demonstration tube and re-train end-to-end with both analytic projections active in the forward pass: a CLF projection for stability and a per-obstacle CBF projection $\nabla h \cdot \mathrm{ReLU}\big(-(\nabla h \cdot f + \alpha_h h)\big)/\|\nabla h\|^2$ for safety, where $h(x) = \sum_i \big((x_i - c_i)/r_i\big)^2 - 1$. Both constraints are enforced in closed form, so no QP is required and the imitation loss can back-propagate through the safety layer. We report results on two shapes – *JShape* and *Sshape* (Figs. 6–5, Tables 5–**??**) — with the same imitation metrics as Section 4.1 plus six safety diagnostics on the rollouts: minimum raw clearance, minimum normalized barrier $h_{\min}$, mean projection correction



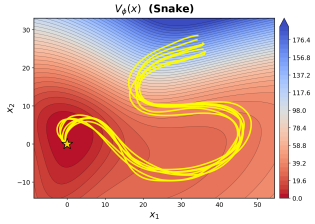
(a) LPV-DS: vector field with reproductions.



(b) LPV-DS: quadratic Lyapunov function $V(x) = (x - x^*)^\top P(x - x^*)$.



(c) Stable NODE (ours): vector field with reproductions.



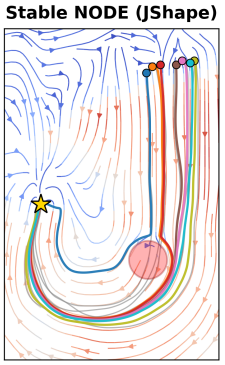
(d) Stable NODE (ours): learned Lyapunov function $V_\phi(x)$.

Figure 3: Qualitative comparison on the LASA *Snake* motion. Top row: LPV-DS [3]; bottom row: ours. Left column shows the learned vector field with rollouts from the demonstration initial conditions; right column shows the corresponding Lyapunov function.

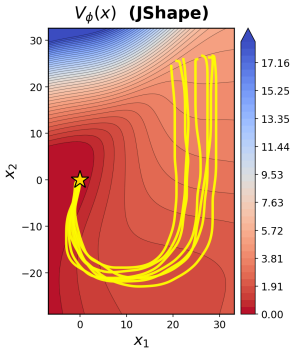
$\|f_{\text{proj}} - f_{\text{net}}\|$ (overall and within $0.2\,r$ of the boundary), arc length inside that band, and in-collision count. Both shapes achieve zero collisions with a strictly positive minimum clearance.

4.2.1. JShape. The demonstration tube curls back over itself, so even a small obstacle forces a noticeable reroute. The seven rollouts in Fig. 6 (a) detour around the obstacle and re-acquire the demo tube within a few steps. All 5454 rollout points are safe (min clearance $= 0.65$ mm, $h_{\min} = 0.398$), and the correction magnitude near the boundary (0.16 mm/s on 0.7% of points) is the same order as the global mean (0.18 mm/s), so the model has internalized the detour and the CBF layer only contributes fine corrections. The 2.41 mm of boundary-following arc indicates a *tight* detour rather than a wide swing. The large DTWD spread (± 5262) is driven by two demos whose initial conditions force a left-side detour, doubling the local shape error on those rollouts.

4.2.2. Sshape. The obstacle sits on the middle limb of the "S" and locally severs the demo tube; the rollouts in Fig. 5 (a) bend above the obstacle and re-acquire the bottom limb. The detour is wider than for JShape: no rollout point enters the $0.2\,r$ band (mean correction near obstacle $= 0$, boundary-following arc $= 0$), with min clearance 1.15 mm and $h_{\min} = 0.614$. The learned dynamics preemptively curve away, so the CBF layer is effectively dormant at deployment. The cost is a larger imitation degradation than for JShape (DTWD 9390 vs. 3766, RMSE 7.39 vs. 5.53 mm/s), the natural price of a more conservative detour. $0/8268$ collisions.

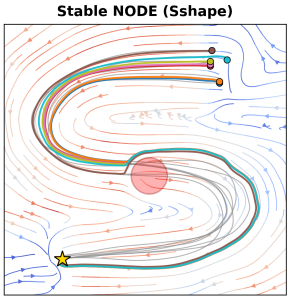


(a) Projected vector field with rollouts.

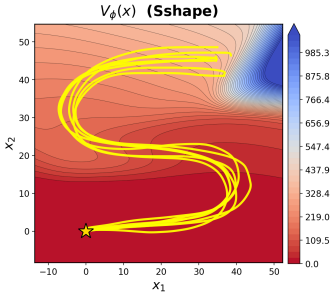


(b) Learned Lyapunov function $V_\phi(x)$.

Figure 4: Stable NODE on the LASA *JShape* motion with one CBF obstacle. Left: rollouts (colored) and projected vector field (CLF + CBF active) successfully detour around the obstacle. Right: corresponding learned Lyapunov function.



(a) Projected vector field with rollouts.



(b) Learned Lyapunov function $V_\phi(x)$.

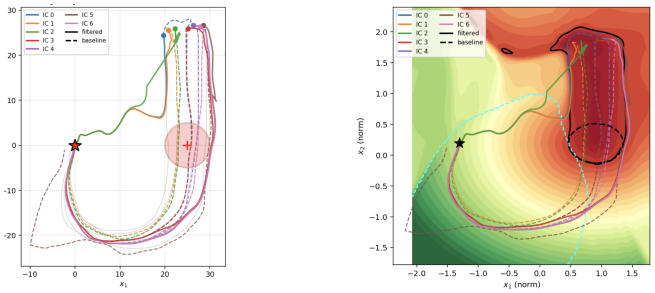
Figure 5: Stable NODE on the LASA *Sshape* motion with one CBF obstacle. Left: rollouts (colored) and projected vector field (CLF + CBF active) detour around the obstacle while still converging to the attractor. Right: corresponding learned Lyapunov function.

4.3. 3D extension

The framework lifts to 3D essentially unchanged: only the input dimension of the MLP $\hat{f}_\theta$ and the warped Lyapunov candidate V_ϕ is widened from 2 to 3, and the CBF barrier $h(x) = \sum_{i=1}^{3} \big((x_i - c_i)/r_i\big)^2 - 1$ now describes an axis-aligned ellipsoid in raw space (still a sphere in normalized coordinates, so the analytic projection of Section 3.3.2 applies verbatim). We evaluate on *3D_Cshape_top* from the phys-gmm benchmark. with one ellipsoidal obstacle placed at the mid-point of the first demo. Results in raw meter units

TABLE 5: Comparison between CBF and HJB Value function based safety correction on the LASA *JShape* motion with one obstacle (7 demos, 7 rollouts). Top block: imitation quality against demonstrations. Bottom block: safety

Metric	Stable NODE (ours)	
Imitation		
DTWD (total, band = 20)	3766.46 ± 5262.25	
RMSE of velocity [mm/s]	5.53	
Mean velocity deviation $\dot{e}$	3.75×10^{-2}	
Safety	**Safety**	**HJB**
Min clearance [mm]	0.65	0.1254
$h_{\min}$ (normalized)	0.398	0.0251
Mean correction near obstacle [mm/s]	0.16	0.6456
Boundary-following arc length [mm]	2.41	3.0576



(a) Projected vector field with rollouts.

(b) Learned Value function $V_{hjb}(x)$.

Figure 6: Stable NODE on the LASA *JShape* motion with one CBF obstacle. We observe usage of the HJB based reachability value function as the safety constraint. Also visualizes the learnt value function for HJB for this NODE.

are reported in Table 6; Figs. 7 and 8 provide the rollouts-only ablation and the six-panel diagnostic.

The rollouts-only ablation (Fig. 7) cleanly isolates the contribution of each layer: raw $\hat{f}$ alone reproduces the C-shape but cuts through the obstacle on every IC, CLF-only already recovers global convergence to the goal but still ignores safety, and only the full CLF + CBF model simultaneously detours around the ellipsoid and reaches the attractor while preserving the C-shape away from the obstacle. The six-panel diagnostic (Fig. 8) confirms that the projection behaves correctly: the projected and raw 3D quivers are visually indistinguishable away from the obstacle (matching the 0.010 m/s global mean correction in Table 6); V_ϕ has elongated, non-ellipsoidal level sets that follow the C-shape; V_ϕ decreases monotonically along every rollout; and the $\|f_{\text{proj}} - \hat{f}\|$ heat-map is essentially zero except in a thin halo of width $\sqrt{1+h_{\text{cap}}}\,r$ around the obstacle, exactly the localization region predicted in Section **??**. Quantitatively all imitation metrics remain tight ($\text{DTWD}_{\text{path-norm}} = 0.21$ m/pair, $\text{RMSE}_v = 0.09$ m/s, trajectory imitation MSE $= 0.068$ m^2) and safety is strict: 0 of 32,016 rollout points violate the barrier, $h_{\min} = 0.40$ corresponds to a radial safety margin of $t_{\min} = 1.18$ (i.e. 18% beyond the obstacle surface), and the CBF contributes only minute corrections except in the immediate vicinity of the boundary (0.137 m/s mean correction on the 0.1% of points inside the $0.2\,r$ band). All conclusions from the 2D evaluation, therefore carry over to 3D without any change to the optimization or model architecture beyond the input dimension.

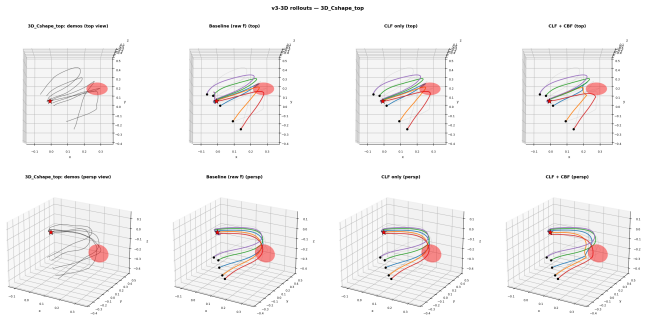


Figure 7: Rollouts on *3D_Cshape_top* with one CBF obstacle. Rows: top-down and perspective views; columns: demos, raw $\hat{f}$, CLF only, CLF + CBF.

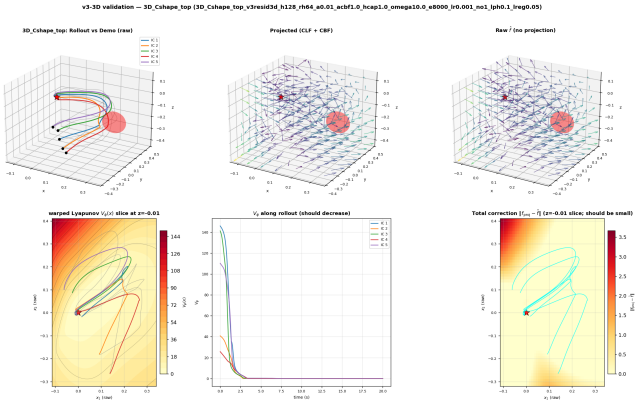


Figure 8: Six-panel diagnostic for *3D_Cshape_top*: rollouts vs. demos, projected and raw vector fields, V_ϕ on the median-z slice, V_ϕ along rollouts, and the correction $\|f_{\text{proj}} - \hat{f}\|$.

4.4. Obstacle avoidance using reachability function

To avoid the obstacle using reachability function, we replace the CBF h function with the reachability based value function $V(x)$ that we learnt. Final constraint applied on the system is:

$$\frac{\partial V_{hjb}}{\partial x}\,\theta^{P}_{K-1,K} z^{P}_{K-1} + \frac{\partial V_{hjb}}{\partial x}\,\theta^{N}_{K-1,K} z^{N}_{K-1} + \alpha_v V_{hjb}(x) \geq 0, \tag{12}$$

5. Implementation Details

The codebase is implemented in Python 3.10 with PyTorch 2.1; autograd flows through the analytic CLF/CBF

TABLE 6: Stable NODE results on the phys-gmm *3D_Cshape_top* motion with one CBF ellipsoidal obstacle (16 demos, 16 rollouts, raw units = meters). Top block: imitation quality against demonstrations. Bottom block: safety / projection diagnostics.

Metric	Stable NODE (ours)
Imitation	
DTWD (total, band = 20)	425.78 ± 41.84
RMSE of velocity [m/s]	0.090
Trajectory imitation MSE [m^2]	0.068 ± 0.011
Safety / CBF	
$h_{\min}$ (normalized)	0.397
$t_{\min} = \sqrt{h_{\min} + 1}$	1.18
Min clearance (conservative) [m]	0.0114
Mean correction $\|f_{\text{proj}} - f_{\text{net}}\|$ [m/s]	0.010
Mean correction near obstacle [m/s]	0.137 (20 / 32016 pts within $0.2\,r$)
Boundary-following arc length [m]	0.059
Violation points	0 / 32016

projection layer. Rollouts use an explicit Euler integrator with step size $\Delta t = 0.02$ and terminate on $\|x_k - x^\star\| < \tau$. f_θ is a 2-layer MLP with hidden dimension 128 and $\tanh$ activations; V_ϕ is a Warped-ICNN (warp dimension 64, ICNN hidden $[64, 64]$, smoothed outer ReLU). Joint single-stage training uses Adam, full-batch over the LASA demos ($\sim$1.5k points per shape), and keeps the checkpoint with lowest $\mathcal{L}_{\text{task}}$. The forward pass during training uses the same $n_{\text{iter}} = 2$ POCS setting as inference (Algorithm 1), so $\mathcal{L}_{\text{task}}$ sees exactly the inference-time field — there is no train/test mismatch in the projection. All experiments in this report were run on a single laptop NVIDIA GPU; end-to-end training of one shape completes in under 10 minutes. Hyperparameters are listed in Table 7.

TABLE 7: Hyperparameters used throughout the experiments (values from the Jshape training command).

Optimization & architecture	
Optimizer / lr / epochs	Adam / 10^{-3} / 2×10^4
f_θ hidden dim, activation	128, $\tanh$
V_ϕ type	Warped-ICNN (64, $[64, 64]$)
Integration step Δt	0.02
Safety & projection	
CLF gain α	0.01
CBF gain α_h	1.0
POCS iterations n_{iter}	2
Bounded barrier cap h_{cap}	1.0
Obstacle radius (normalized)	0.06
Sampling & penalty shaping	
Off-demo tube radius ρ	0.5
CBF penalty box scale β	1.2
Interior cutoff $h_{\min}$	-0.2
Velocity-copy kernel σ	$\rho/2 = 0.25$
Off-demo samples per step n_{off}	512
Loss weights $(\lambda_V, \lambda_H, \lambda_{vc}, \lambda_{sm})$	(0.5, 0.1, 0.5, 0)
Active-projection masking	on
Penalty warmup T_{wu}	2000

The repository[1] is organized as follows.

- `lasa/v2/` — main 2D framework: model, training, validation.
  - `model_safe.py` — `SafeNeuralODE` with analytic CLF/CBF projection layer.
  - `train_inline.py` — joint single-stage training script.
  - `validate.py` — six-panel validation (rollouts, fields, V contour, correction magnitude).
- `lasa/3d/` — 3D extension: residual MLP and 3D Warped-ICNN.

6. Conclusions

We presented a framework for learning Neural ODE policies that are stable to a goal and safe around obstacles. The key observation is geometric: both the CLF and CBF conditions are linear in f, so their feasible sets are half-spaces and the projection onto each is closed-form. Alternating the two projections (POCS) replaces the per-step QP of prior work [6], [7], [8] with a one-line, autograd-friendly update. Training *through* the projection removes any train/test mismatch, and the residual variant of Section 3.5 inherits CLF stability exactly outside the obstacle's cap region. On LASA, our method beats LPV-DS on DTWD and velocity RMSE on every shape tested, with zero CBF violations across all evaluated rollouts. The framework extends to 3D without architectural changes.

6.1. Future work

A neural-HJB variant that would lift the reachability barrier into SE3 space and enable manipulator-scale safety. We also plan to show its extension to various DS methodologies in literature.

We plan to extend this work to linkage-avoidance as well where the collision avoidance is done for each link instead of the end-effector only.

Acknowledgments

We would like to thank Prof. Nadia Figueroa for her instruction in MEAM 6230 *Learning And Control For Adaptive And Reactive Robots*, and the teaching team — Yifan Xue, Mateusz Jaszczuk, and Kabir Puri — for their guidance throughout the semester.

References

[1] A. Billard, S. Calinon, R. Dillmann, and S. Schaal, "Robot programming by demonstration," in *Springer Handbook of Robotics*, 2008, pp. 1371–1394.

1. https://github.com/Skylar-xty/InvarianceNODE/tree/final_project

[2] S. M. Khansari-Zadeh and A. Billard, "Learning stable nonlinear dynamical systems with Gaussian mixture models," *IEEE Trans. Robotics*, vol. 27, no. 5, pp. 943–957, 2011.

[3] N. Figueroa and A. Billard, "A physically-consistent Bayesian non-parametric mixture model for dynamical system learning," in *Proc. CoRL*, 2018.

[4] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, "Neural ordinary differential equations," in *Proc. NeurIPS*, 2018.

[5] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety-critical systems," *IEEE Trans. Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.

[6] W. Xiao, T.-H. Wang, R. Hasani, M. Lechner, Y. Ban, C. Gan, and D. Rus, "On the forward invariance of neural ODEs," in *Proc. ICML*, 2023, arXiv:2210.04763.

[7] F. Nawaz, T. Li, N. Matni, and N. Figueroa, "Learning complex motion plans using neural ODEs with safety and stability guarantees," in *Proc. ICRA*, 2024, arXiv:2308.00186.

[8] K. Miao, S. Zhao, Y. Wang, K. Gatsis, and A. Papachristodoulou, "Opt-ODENet: A neural ODE framework with differentiable QP layers for safe and stable control design," in *Proc. L4DC*, 2025, arXiv:2504.17139.

[9] G. Manek and J. Z. Kolter, "Learning stable deep dynamics models," in *Proc. NeurIPS*, 2019.

[10] B. Amos, L. Xu, and J. Z. Kolter, "Input convex neural networks," in *Proc. ICML*, 2017.

[11] J. J. Choi, D. Lee, K. Sreenath, C. J. Tomlin, and S. L. Herbert, "Robust control barrier-value functions for safety-critical control," in *Proc. CDC*, 2021.

[12] J. P. Boyle and R. L. Dykstra, "A method for finding projections onto the intersection of convex sets in Hilbert spaces," in *Lecture Notes in Statistics*, vol. 37, Springer, 1986, pp. 28–47.

[13] J. von Neumann, *Functional Operators, Vol. II: The Geometry of Orthogonal Spaces*, Annals of Mathematics Studies, vol. 22. Princeton, NJ: Princeton Univ. Press, 1950.

[14] H. H. Bauschke and J. M. Borwein, "On projection algorithms for solving convex feasibility problems," *SIAM Rev.*, vol. 38, no. 3, pp. 367–426, 1996.